POLY PMNA · DIPLOMA CURRICULUM REVISION 2026

Course 5051 | Semester V | Theory | 4 Credits

Software Engineering

A complete syllabus-aligned handbook with clear explanations, practical or exam guidance, Malayalam-support notes, diagrams, activities and revision tools.

Programmes	**Computer Science & Engineering**
Contact hours	**60**
Teaching scheme	**3:1:0:0**
Credits	**4**
CIE / ESE	**Theory CIE 40 / Theory ESE 60**
Self-learning	**5.5 h/week**

CURRICULUM INTEGRITY

Official Source Declaration and Verified Inconsistencies

The attached Revision 2026 index identifies Course 5051 as Software Engineering in Semester V for Computer Science & Engineering. The official SITTTR detailed course page was checked at the indexed link and returned “Requested file not found.” With the user's approved public-source approach, this handbook is transparently based on NPTEL IIT Kharagpur’s Software Engineering course and polytechnic software engineering curricula. It does not claim to reproduce official REV2026 detailed-syllabus wording.

Source treatment: Teaching scheme, credits and assessment values are provisional placeholders aligned to neighbouring handbook format. Software designs, SRS documents, test plans and security protocols must be based on approved enterprise architectures, certified secure coding standards, current industry best practices and competent professional review.

Verified source note: Verified sources support SDLC models, requirement engineering, UML diagrams (Use Case, Sequence, Class), and testing strategies. The four-module sequence is a learning path, not an asserted official module split.

COURSE DASHBOARD

What You Are Expected to Learn

60
Contact hours

4
Credits

Theory CIE 40
CIE

Theory ESE 60
ESE

Course introduction

Software Engineering studies the systematic, disciplined and quantifiable approach to the development, operation and maintenance of software. It develops the ability to analyze software process models (Waterfall, Agile), evaluate requirement specifications (SRS), design software architectures using UML, and coordinate testing and quality assurance while respecting the technical and security boundaries of enterprise software development.

> **Malayalam support:** □ handbook □□□□□□□□□□□□ syllabus topic □□□□□ □□□□□□□□□□; □□□□□□□□ principle, diagram/procedure, application, common mistake □□□□□□ □□□□□□□□ □□□□□□□□.

Prerequisite foundation

C programming, object-oriented programming, data structures and basic database management systems.

Use prerequisites only as a revision bridge. The current course outcomes and detailed syllabus define the required performance.

Teaching and assessment scheme

L	T	P	S	Self-learning/week	Contact hours	Credits	CIE	ESE
3	1	0	0	5.5 h/week	60	4	Theory CIE 40	Theory ESE 60

STUDY METHOD

How to Use This Handbook

1 · Understand

Read terms, conditions and purpose; make a short definition in your own words.

2 · Visualise

Follow the diagram, circuit, flow or procedure and label it accurately.

3 · Apply

Use the method block, calculation or practical checklist without skipping units or safety checks.

4 · Revise

Use questions, quick cards and common-mistake notes before examination.

OFFICIAL STATEMENTS

Objectives and Course Outcomes

Official course objectives

1. Explain the software development life cycle (SDLC) and various process models including Waterfall, Spiral and Agile.
2. Apply requirement engineering techniques to gather, analyze and document functional and non-functional requirements.
3. Design software systems using modular principles, architectural patterns and Unified Modeling Language (UML) diagrams.
4. Apply systematic testing techniques and quality assurance standards to ensure software reliability and maintainability.

Student-friendly meaning

You should be able to recognise the relevant system or material, explain its principle, communicate through a correct diagram/table where needed and follow a defensible solution or practical method.

> **Malayalam support:** CO □□□□□□ exam-□ □□□□□ □□□□□□□ □□□□□□ □□□□□ □□□□□□□□.
> Define, explain, apply □□□□□ action words □□□□□□□□□□□□□.

Official Course Outcomes

CO	Official outcome	Bloom level
CO1	Explain software process models and the phases of the software development life cycle.	Understand
CO2	Develop Software Requirement Specifications (SRS) for given application scenarios.	Apply
CO3	Design software systems using UML diagrams and modular design principles.	Apply
CO4	Apply black-box and white-box testing techniques for software verification and validation.	Apply

Official CO–PO articulation matrix

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7–PO11
CO1	3	2	2	2	2	–	–
CO2	3	2	2	2	2	–	–
CO3	3	2	2	2	2	–	–
CO4	3	2	2	2	2	–	–

SYLLABUS COVERAGE

Curriculum Coverage Map

All official major topics mapped to handbook chapters

Module	Title	Official major topics	Hours	CO	Handbook section
1	Software Process Models	Software characteristics and myths; Software Development Life Cycle (SDLC); Waterfall model; Spiral model; Prototyping; Agile methodologies (Scrum, Extreme Programming); selection of process models.	Approx. 14 hours	CO1	Module 1
2	Requirement Engineering	Requirement gathering and analysis; functional and non-functional requirements; Software Requirement Specification (SRS) structure; IEEE standards for SRS; requirement validation and management.	Approx. 14 hours	CO2	Module 2
3	Software Design and UML	Design principles (modularity, abstraction, information hiding); Cohesion and Coupling; Architectural patterns (Client-Server, Layered); Unified Modeling Language (UML): Use Case, Class, Sequence and State diagrams.	Approx. 16 hours	CO3	Module 3
4	Coding, Testing and Maintenance	Coding standards and reviews; Testing fundamentals (Verification vs Validation); Black-box testing (Equivalence partitioning, Boundary value analysis); White-box testing (Path testing, MC/DC); Software maintenance and CMMI.	Apply	CO4	Module 4

LEARNING ROADMAP

How the Modules Connect

Software Process Models

Software characteristics and myths; Software Development Life Cycle (SDLC); Waterfall model; Spiral model; Prototyping; Agile methodologies (Scrum, Extreme Programming); selection of process models.

CO1 · Approx. 14 hours

Requirement Engineering

Requirement gathering and analysis; functional and non-functional requirements; Software Requirement Specification (SRS) structure; IEEE standards for SRS; requirement validation and management.

CO2 · Approx. 14 hours

Software Design and UML

Design principles (modularity, abstraction, information hiding); Cohesion and Coupling; Architectural patterns (Client-Server, Layered); Unified Modeling Language (UML): Use Case, Class, Sequence and State diagrams.

CO3 · Approx. 16 hours

Coding, Testing and Maintenance

Coding standards and reviews; Testing fundamentals (Verification vs Validation); Black-box testing (Equivalence partitioning, Boundary value analysis); White-box testing (Path testing, MC/DC); Software maintenance and CMMI.

CO4 · Apply

MODULE 1

Software Process Models

Software characteristics and myths; Software Development Life Cycle (SDLC); Waterfall model; Spiral model; Prototyping; Agile methodologies (Scrum, Extreme Programming); selection of process models.

Approx. 14 hours **CO1** **Understand · Apply**

Learning goals

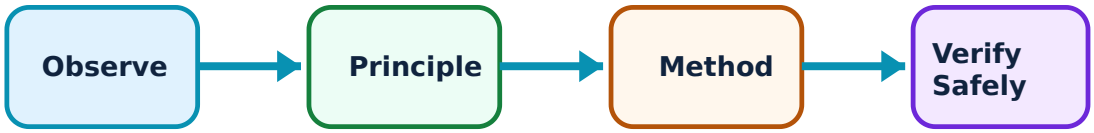
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Software Process Models: identify the system, state its principle, follow the correct method and verify the result safely.

1. SDLC and traditional models−

The SDLC provides a structured framework for software development. The Waterfall model is linear and sequential, suitable for well-defined requirements. The Spiral model integrates risk analysis. Prototyping allows for early user feedback. Understanding these models is essential for choosing the right approach for a given project.

Study and application method

List the phases of the 'Waterfall Model' and explain its limitations; describe the 'Spiral Model' and its focus on risk management.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: List the phases of the 'Waterfall Model' and explain its limitations; describe the 'Spiral Model' and its focus on risk management.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Process models are frameworks, not rigid rules. Do not apply the Waterfall model to projects with highly volatile requirements without authorised risk mitigation.

2. Agile methodologies–

Agile focuses on iterative development, customer collaboration and responding to change. Scrum is a popular Agile framework using short 'Sprints'. Agile is ideal for dynamic environments where requirements evolve, emphasizing working software over comprehensive documentation.

Study and application method

Explain the core values of the 'Agile Manifesto'; describe the role of the 'Scrum Master' and the 'Product Owner' in a Scrum team.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Explain the core values of the 'Agile Manifesto'; describe the role of the 'Scrum Master' and the 'Product Owner' in a Scrum team.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Agile requires high team discipline and customer involvement. Do not adopt Agile without authorised training and a commitment to frequent communication and feedback.

Module 1 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 2

Requirement Engineering

Requirement gathering and analysis; functional and non-functional requirements; Software Requirement Specification (SRS) structure; IEEE standards for SRS; requirement validation and management.

Approx. 14 hours | CO2 | Understand · Apply

Learning goals

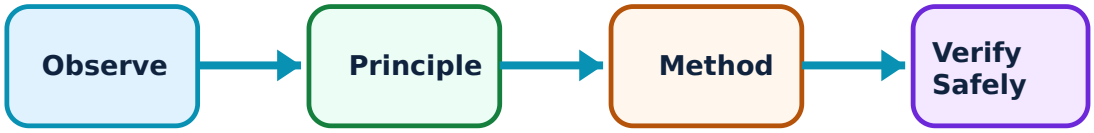
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Requirement Engineering: identify the system, state its principle, follow the correct method and verify the result safely.

1. Requirement analysis and SRS−

Requirement engineering involves eliciting user needs and documenting them in an SRS. Functional requirements define what the system does, while non-functional requirements (e.g., performance, security) define how it performs. A well-structured SRS is the foundation for all subsequent development phases.

Study and application method

Differentiate between 'Functional' and 'Non-functional' requirements with examples; list the characteristics of a good SRS document.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Differentiate between 'Functional' and 'Non-functional' requirements with examples; list the characteristics of a good SRS document.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Ambiguous requirements lead to project failure. Do not proceed to design without authorised SRS sign-off from all stakeholders.

2. Requirement validation−

Validation ensures that the requirements accurately reflect user needs and are feasible. Techniques include reviews, walkthroughs and prototyping. Requirement management tracks changes throughout the project lifecycle to prevent scope creep.

Study and application method

Describe the process of 'Requirement Validation'; explain the concept of 'Scope Creep' and how it can be managed conceptually.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Describe the process of 'Requirement Validation'; explain the concept of 'Scope Creep' and how it can be managed conceptually.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Requirements change over time. Ensure all changes follow an authorised change-control process to maintain project alignment and budget.

Module 2 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 3

Software Design and UML

Design principles (modularity, abstraction, information hiding); Cohesion and Coupling; Architectural patterns (Client-Server, Layered); Unified Modeling Language (UML): Use Case, Class, Sequence and State diagrams.

Approx. 16 hours | CO3 | Understand · Apply

Learning goals

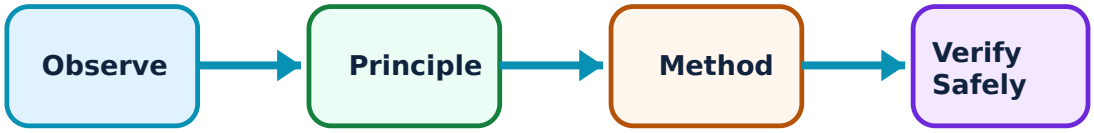
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Software Design and UML: identify the system, state its principle, follow the correct method and verify the result safely.

1. Design principles and modularity−

Effective software design uses modularity to manage complexity. Cohesion measures how focused a module is (high cohesion is good), while Coupling measures the interdependency between modules (low coupling is good). Information hiding protects internal data from unauthorized access.

Study and application method

Explain the terms 'Cohesion' and 'Coupling' in the context of modular design; describe the 'Layered Architecture' pattern.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Explain the terms 'Cohesion' and 'Coupling' in the context of modular design; describe the 'Layered Architecture' pattern.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Poor design leads to 'Spaghetti Code' that is hard to maintain. All software architectures must follow authorised design patterns and coding standards.

2. UML modeling−

UML provides graphical notations to model software systems. Use Case diagrams show system functionality from a user's perspective. Class diagrams show the static structure and relationships. Sequence diagrams show dynamic interactions over time. These models bridge the gap between requirements and implementation.

Study and application method

Sketch a 'Use Case Diagram' for a simple Library Management System; explain the purpose of a 'Sequence Diagram' in software design.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Sketch a 'Use Case Diagram' for a simple Library Management System; explain the purpose of a 'Sequence Diagram' in software design.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: UML diagrams must be kept consistent with the actual implementation. Do not use outdated models for system maintenance or debugging without authorised verification.

Module 3 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 4

Coding, Testing and Maintenance

Coding standards and reviews; Testing fundamentals (Verification vs Validation); Black-box testing (Equivalence partitioning, Boundary value analysis); White-box testing (Path testing, MC/DC); Software maintenance and CMMI.

Apply | CO4 | Understand · Apply

Learning goals

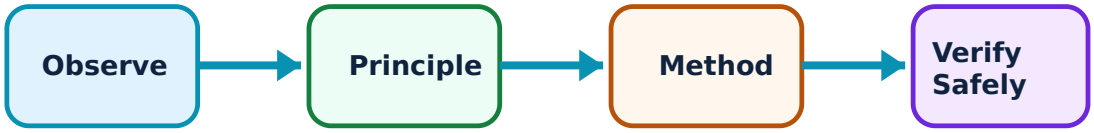
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Coding, Testing and Maintenance: identify the system, state its principle, follow the correct method and verify the result safely.

1. Software testing techniques–

Testing ensures software quality. Black-box testing focuses on inputs and outputs without knowing internal code. White-box testing examines the internal logic and paths. MC/DC (Modified Condition/Decision Coverage) is a rigorous technique used in safety-critical systems.

Study and application method

Compare 'Black-box' and 'White-box' testing in a conceptual table; explain 'Boundary Value Analysis' with a conceptual example.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Compare 'Black-box' and 'White-box' testing in a conceptual table; explain 'Boundary Value Analysis' with a conceptual example.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Testing cannot prove the absence of bugs, only their presence. Do not release software without authorised completion of the test plan and security audits.

2. Maintenance and quality−

Software maintenance involves fixing bugs, adapting to new environments and adding features. Quality models like CMMI (Capability Maturity Model Integration) help organizations improve their development processes and deliver reliable software products.

Study and application method

List the four types of software maintenance (Corrective, Adaptive, Perfective, Preventive); explain the significance of 'CMMI Levels'.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: List the four types of software maintenance (Corrective, Adaptive, Perfective, Preventive); explain the significance of 'CMMI Levels'.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Maintenance is the most expensive phase of the software lifecycle. Ensure all code is documented and follows authorised version-control and configuration-management protocols.

Module 4 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

FORMULA AND PROCEDURE BANK

Rapid Recall Bank

Recall 1

State symbols and SI units before substitution.

Recall 2

Draw the circuit, system boundary, vector or process flow before reasoning.

Recall 3

> For laboratory work: aim → apparatus → diagram → procedure → observation → calculation → result → precautions.

Recall 4

> For a graph: label axes, units, scale, operating region and conclusion.

Recall 5

> For a comparison: use construction, working, result, advantage and limitation.

Recall 6

> For an extended answer: definition/principle, labelled diagram, working steps, application and a caution/limitation.

DIAGRAM LIBRARY

Diagram Library for Rapid Revision

Module 1: Software Process Models

Use the concept flow to revise observation, principle, method and verification.

Module 2: Requirement Engineering

Use the concept flow to revise observation, principle, method and verification.

Module 3: Software Design and UML

Use the concept flow to revise observation, principle, method and verification.

Module 4: Coding, Testing and Maintenance

Use the concept flow to revise observation, principle, method and verification.

SELF-LEARNING

Official Self-Learning and Student Activities

Structured activity

Compare Waterfall and Agile models in a conceptual table showing three differences.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Develop a Software Requirement Specification (SRS) outline for a simple E-commerce application.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Sketch a Class Diagram for a 'Bank Account' system showing inheritance and association.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Apply Boundary Value Analysis conceptually to test an input field that accepts ages from 18 to 60.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Identify the software development model used by a popular tech company or an open-source project.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Activity discipline: The supplied PDF assigns the listed self-learning hours. This handbook preserves the activity direction; the teacher’s approved schedule and safety instructions govern execution.

EXAMINATION READINESS

Assessment Methodology and Examination Pattern

Official assessment structure		
Component	**Official mark / conversion**	**Student evidence**
Attendance	5 marks	End-of-semester attendance component.
Self-learning activities	15 marks	Module-linked activities.
Series examinations	20 + 20 converted	Two 50-mark series examinations.
CIE total	Theory CIE 40	Continuous internal evaluation.
Written ESE	Theory ESE 60	2.5 hours; official Part A / Part B / Part C pattern.

Series-test pattern

The supplied theory pattern uses Part A (1-mark), Part B (3-mark with choice) and Part C (7-mark) distribution across the stated modules. Practical courses follow the supplied practical-test scheme.

Examination evidence

Show a complete method, correct units/conditions, clear diagrams or tables, a result/conclusion and safety awareness for any apparatus or process involved.

EXAM PRACTICE

Module-wise Questions with Answers

Module 1 — Software Process Models

Explain SDLC and traditional models and state one correct method, application or precaution.–

The SDLC provides a structured framework for software development. The Waterfall model is linear and sequential, suitable for well-defined requirements. The Spiral model integrates risk analysis. Prototyping allows for early user feedback. Understanding these models is essential for choosing the right approach for a given project.

Answer framework: List the phases of the 'Waterfall Model' and explain its limitations; describe the 'Spiral Model' and its focus on risk management.

Important point: Process models are frameworks, not rigid rules. Do not apply the Waterfall model to projects with highly volatile requirements without authorised risk mitigation.

Explain Agile methodologies and state one correct method, application or precaution.–

Agile focuses on iterative development, customer collaboration and responding to change. Scrum is a popular Agile framework using short 'Sprints'. Agile is ideal for dynamic environments where requirements evolve, emphasizing working software over comprehensive documentation.

Answer framework: Explain the core values of the 'Agile Manifesto'; describe the role of the 'Scrum Master' and the 'Product Owner' in a Scrum team.

Important point: Agile requires high team discipline and customer involvement. Do not adopt Agile without authorised training and a commitment to frequent communication and feedback.

Module 2 — Requirement Engineering

Explain Requirement analysis and SRS and state one correct method, application or precaution.–

Requirement engineering involves eliciting user needs and documenting them in an SRS. Functional requirements define what the system does, while non-functional requirements (e.g., performance, security) define how it performs. A well-structured SRS is the foundation for all subsequent development phases.

Answer framework: Differentiate between 'Functional' and 'Non-functional' requirements with examples; list the characteristics of a good SRS document.

Important point: Ambiguous requirements lead to project failure. Do not proceed to design without authorised SRS sign-off from all stakeholders.

Explain Requirement validation and state one correct method, application or precaution.–

Validation ensures that the requirements accurately reflect user needs and are feasible. Techniques include reviews, walkthroughs and prototyping. Requirement management tracks changes throughout the project lifecycle to prevent scope creep.

Answer framework: Describe the process of 'Requirement Validation'; explain the concept of 'Scope Creep' and how it can be managed conceptually.

Important point: Requirements change over time. Ensure all changes follow an authorised change-control process to maintain project alignment and budget.

Module 3 — Software Design and UML

Explain Design principles and modularity and state one correct method, application or precaution.–

Effective software design uses modularity to manage complexity. Cohesion measures how focused a module is (high cohesion is good), while Coupling measures the interdependency between modules (low coupling is good). Information hiding protects internal data from unauthorized access.

Answer framework: Explain the terms 'Cohesion' and 'Coupling' in the context of modular design; describe the 'Layered Architecture' pattern.

Important point: Poor design leads to 'Spaghetti Code' that is hard to maintain. All software architectures must follow authorised design patterns and coding standards.

Explain UML modeling and state one correct method, application or precaution.–

UML provides graphical notations to model software systems. Use Case diagrams show system functionality from a user's perspective. Class diagrams show the static structure and relationships. Sequence diagrams show dynamic interactions over time. These models bridge the gap between requirements and implementation.

Answer framework: Sketch a 'Use Case Diagram' for a simple Library Management System; explain the purpose of a 'Sequence Diagram' in software design.

Important point: UML diagrams must be kept consistent with the actual implementation. Do not use outdated models for system maintenance or debugging without authorised verification.

Module 4 — Coding, Testing and Maintenance

Explain Software testing techniques and state one correct method, application or precaution.–

Testing ensures software quality. Black-box testing focuses on inputs and outputs without knowing internal code. White-box testing examines the internal logic and paths. MC/DC (Modified Condition/Decision Coverage) is a rigorous technique used in safety-critical systems.

Answer framework: Compare 'Black-box' and 'White-box' testing in a conceptual table; explain 'Boundary Value Analysis' with a conceptual example.

Important point: Testing cannot prove the absence of bugs, only their presence. Do not release software without authorised completion of the test plan and security audits.

Explain Maintenance and quality and state one correct method, application or precaution.–

Software maintenance involves fixing bugs, adapting to new environments and adding features. Quality models like CMMI (Capability Maturity Model Integration) help organizations improve their development processes and deliver reliable software products.

Answer framework: List the four types of software maintenance (Corrective, Adaptive, Perfective, Preventive); explain the significance of 'CMMI Levels'.

Important point: Maintenance is the most expensive phase of the software lifecycle. Ensure all code is documented and follows authorised version-control and configuration-management protocols.

PRACTICE ONLY

Practice Model Paper

Important: This practice paper is based on the official pattern in the supplied syllabus.

It is **not** an official SITTTR model question paper.

Part A — short response

1. State one key definition from Module 1: Software Process Models.
2. Identify one diagram, observation, formula or safety condition relevant to Module 1.
3. State one key definition from Module 2: Requirement Engineering.
4. Identify one diagram, observation, formula or safety condition relevant to Module 2.
5. State one key definition from Module 3: Software Design and UML.
6. Identify one diagram, observation, formula or safety condition relevant to Module 3.
7. State one key definition from Module 4: Coding, Testing and Maintenance.
8. Identify one diagram, observation, formula or safety condition relevant to Module 4.

Part B — structured explanation

1. Explain a selected procedure/principle from Module 1 with a labelled diagram, table or method.
2. Explain a selected procedure/principle from Module 2 with a labelled diagram, table or method.
3. Explain a selected procedure/principle from Module 3 with a labelled diagram, table or method.
4. Explain a selected procedure/principle from Module 4 with a labelled diagram, table or method.

Part C — extended response / practical task

1. Develop a complete answer or practical plan for: Software characteristics and myths; Software Development Life Cycle (SDLC); Waterfall model; Spiral model; Prototyping; Agile methodologies (Scrum, Extreme Programming); selection of process models..
2. Develop a complete answer or practical plan for: Requirement gathering and analysis; functional and non-functional requirements; Software Requirement Specification (SRS) structure; IEEE standards for SRS; requirement validation and management..
3. Develop a complete answer or practical plan for: Design principles (modularity, abstraction, information hiding); Cohesion and Coupling; Architectural patterns (Client-Server, Layered); Unified Modeling Language (UML): Use Case, Class, Sequence and State diagrams..
4. Develop a complete answer or practical plan for: Coding standards and reviews; Testing fundamentals (Verification vs Validation); Black-box testing (Equivalence partitioning, Boundary value analysis); White-box testing (Path testing, MC/DC); Software maintenance and CMMI..

LAST-MILE REVISION

Quick Revision

Module 1: Software Process Models

Software characteristics and myths; Software Development Life Cycle (SDLC); Waterfall model; Spiral model; Prototyping; Agile methodologies (Scrum, Extreme Programming); selection of process models.

- Match each topic to CO1.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 2: Requirement Engineering

Requirement gathering and analysis; functional and non-functional requirements; Software Requirement Specification (SRS) structure; IEEE standards for SRS; requirement validation and management.

- Match each topic to CO2.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 3: Software Design and UML

Design principles (modularity, abstraction, information hiding); Cohesion and Coupling; Architectural patterns (Client-Server, Layered); Unified Modeling Language (UML): Use Case, Class, Sequence and State diagrams.

- Match each topic to CO3.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 4: Coding, Testing and Maintenance

Coding standards and reviews; Testing fundamentals (Verification vs Validation); Black-box testing (Equivalence partitioning, Boundary value analysis); White-box testing (Path testing, MC/DC); Software maintenance and CMMI.

- Match each topic to CO4.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Common mistakes: Writing an unlabelled diagram; ignoring units, polarity or conditions; treating a practical result as valid without observations; confusing a definition with an application; and omitting a safety precaution where the topic uses apparatus, current, heat, chemicals or tools.

Exam checklist: Read the command word; answer every part; draw clean labelled diagrams; use a clear calculation/procedure; state result with units or observation; and reserve two minutes for checking.

VOCABULARY

Glossary

Important terms from the syllabus	
Term / topic	**One-line meaning**
SDLC and traditional models	The SDLC provides a structured framework for software development.
Agile methodologies	Agile focuses on iterative development, customer collaboration and responding to change.
Requirement analysis and SRS	Requirement engineering involves eliciting user needs and documenting them in an SRS.
Requirement validation	Validation ensures that the requirements accurately reflect user needs and are feasible.
Design principles and modularity	Effective software design uses modularity to manage complexity.
UML modeling	UML provides graphical notations to model software systems.
Software testing techniques	Testing ensures software quality.
Maintenance and quality	Software maintenance involves fixing bugs, adapting to new environments and adding features.

LEARNING RESOURCES

References

Recommended software-engineering references

1. Rajib Mall, Fundamentals of Software Engineering.
2. Roger S. Pressman, Software Engineering: A Practitioner's Approach.
3. Ian Sommerville, Software Engineering.
4. Pankaj Jalote, An Integrated Approach to Software Engineering.

Approved public software-engineering sources

- https://nptel.ac.in/courses/106105182
- https://onlinecourses.nptel.ac.in/noc20_cs68/preview

These sources provide the verified study basis while official Course 5051 detail is unavailable; they do not replace official enterprise designs, authorised secure coding standards, certified deployment pipelines or authorised IT service plans.